Faculty of Information Technology
20241

ASPU
Al-Sham Private University

Multimedia-Lecture-Four

Images

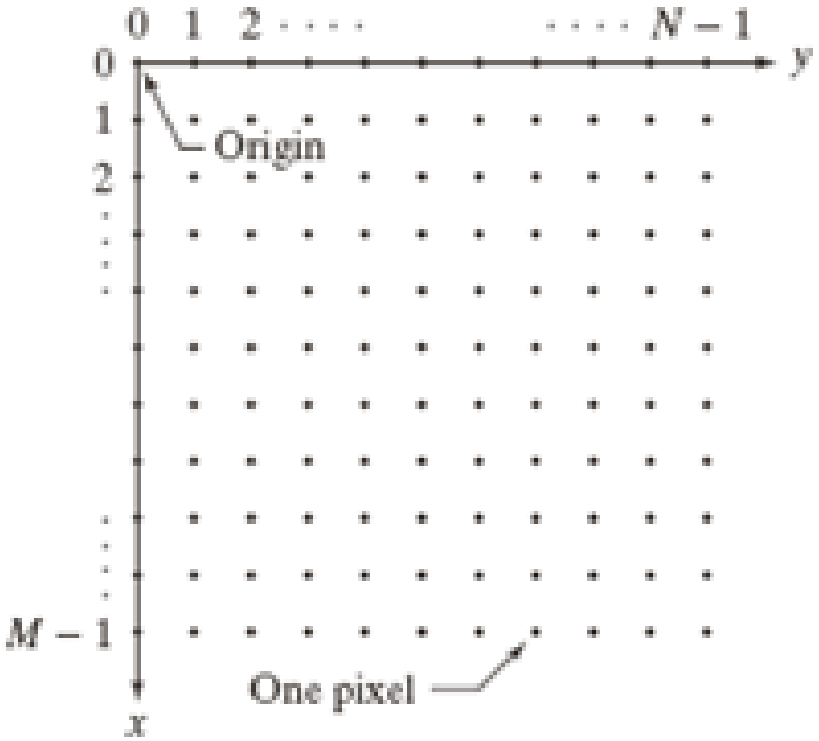
Prof. Houssam AL-Fawal
Eng. Noor AL-Hakim

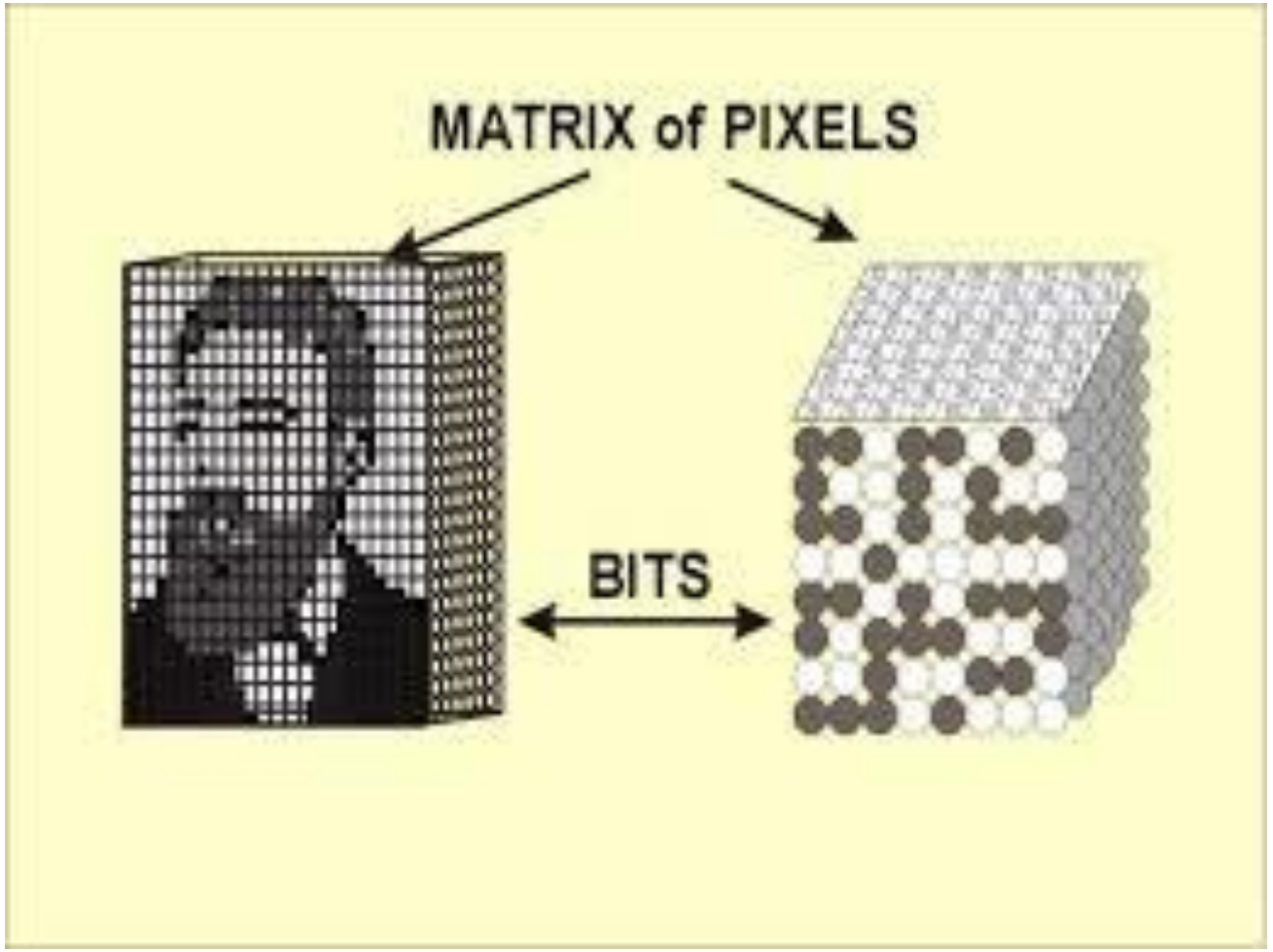
Multimedia-Lecture-Four

Images

Eng. Noor AL-Hakim

Digital Image Representation





Dealing With Image in PYTHON

- Reading Image
- Displaying Image
- Writing Image
- Find More Information About Image:
  - Whos Function
  - Imfinfo Function

Reading image:

```
>> I = cv2.imread();
>> r, c = I.shape[:2]
```

Shape function returns rows (r) and columns (c) of an image.

Displaying Image:

```
>> cv2.imshow('Image', I)
```

Min and Max value of pixel:

```
>> min_value = I.min()
>> max_value = I.max()
```

Find More Information About Image:

Some functions displays additional information about an array for example size, class, …etc.

```
>> print(I.dtype)
>> print(I.size)
```

Writing Image:

```
>> cv2.imwrite(‘filename.jpg’, I)
```

Note: For JPEG Images only, you can apply:

```
cv2.imwrite('filename.jpg',[int(cv2.IMWRITE_JPEG_QUALITY), q])
```

where q is an integer between 0 and 100

Image Format

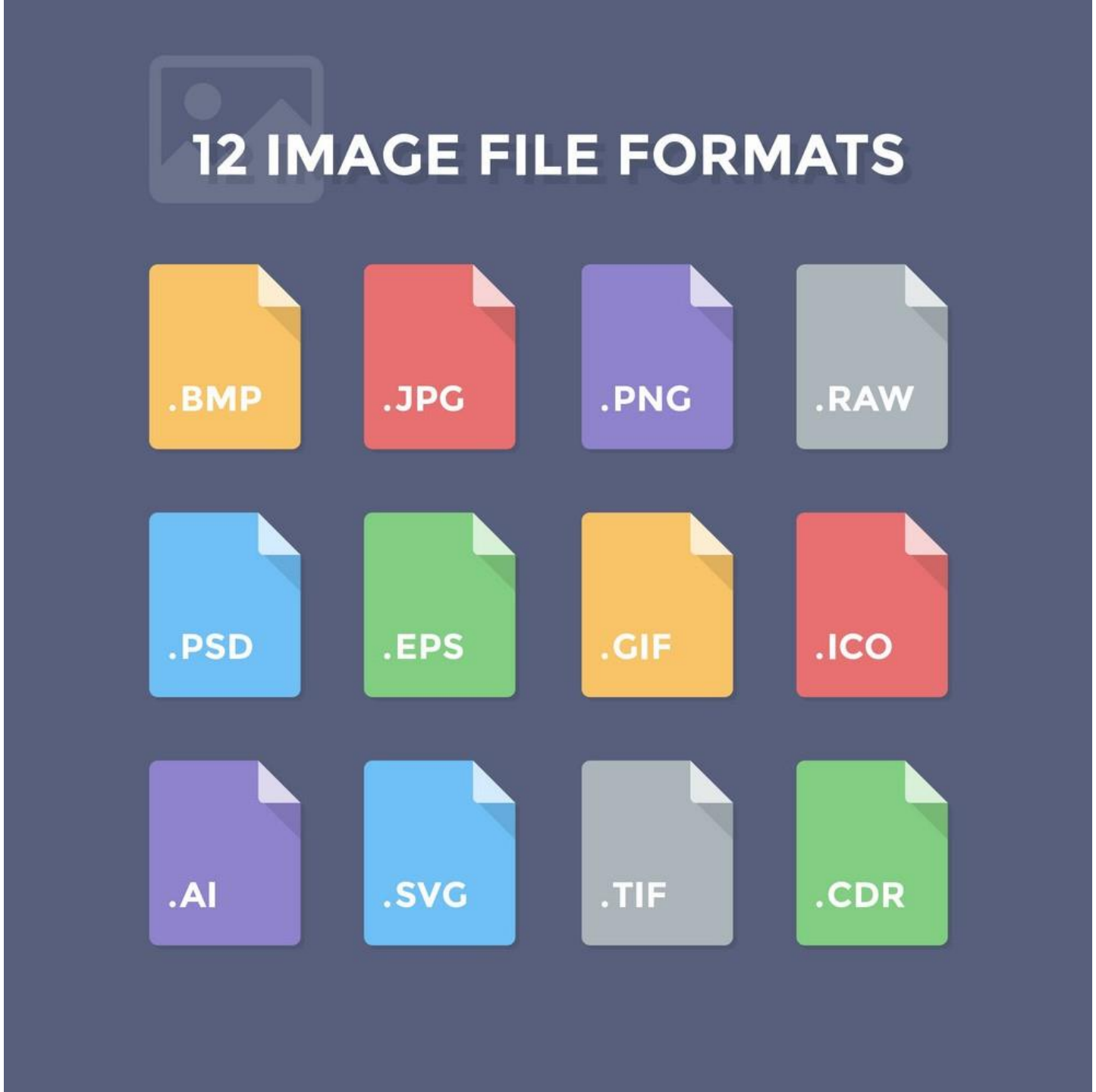


Image Data Classes

Name	Description
`double`	Double-precision, floating-point numbers in the approximate range -10^{308} to 10^{308} (8 bytes per element).
`uint8`	Unsigned 8-bit integers in the range [0, 255] (1 byte per element).
`uint16`	Unsigned 16-bit integers in the range [0, 65535] (2 bytes per element).
`uint32`	Unsigned 32-bit integers in the range [0, 4294967295] (4 bytes per element).
`int8`	Signed 8-bit integers in the range [−128, 127] (1 byte per element).
`int16`	Signed 16-bit integers in the range [−32768, 32767] (2 bytes per element).
`int32`	Signed 32-bit integers in the range [−2147483648, 2147483647] (4 bytes per element).
`single`	Single-precision floating-point numbers with values in the approximate range -10^{38} to 10^{38} (4 bytes per element).
`char`	Characters (2 bytes per element).
`logical`	Values are 0 or 1 (1 byte per element).

Image Types

- Binary images.
- Intensity (gray-level) images.
- Color images.
- Indexed images.

Binary Images (1-Bit)

Binary images are images whose pixels have only two possible intensity values.

Converting RGB image to binary image:

```
# Read the image
I = cv2.imread('home.jpg',0)

# Convert the image from RGB to binary (B&W)
_, J = cv2.threshold(I, 127, 255, cv2.THRESH_BINARY)

# Display the output image
cv2.imshow('BW',J)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Intensity Images (Gray-Scale 8-Bits)

When an element of an image are of class uint8 and uint16, they have

integer values in the range [0,255] and [0,65535] respectively.

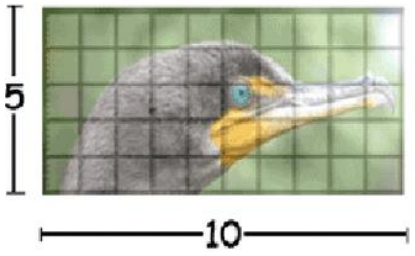
Elements of class double are in range [0,1].

Converting RGB image to gray-scale image:

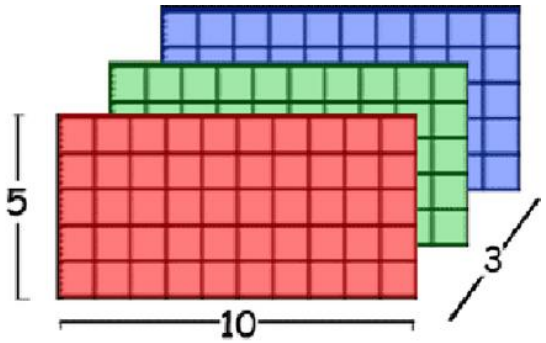
```
# Read the image as Gray-Scale (Method1)
gray = cv2.imread('home.jpg',0)
# Display the output image
cv2.imshow('Gray', gray )
cv2.waitKey(0)
cv2.destroyAllWindows()
################################
I = cv2.imread('home.jpg')
# Convert the image to Gray-Scale (Method2)
gray =cv2.cvtColor(I, cv2.COLOR_RGB2GRAY)
cv2.imshow('Gray', gray )
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Color Images (24-Bits):

An RGB image is an MxNx3 array of color pixels where each color pixel is a triplet corresponding to the red, green, and blue components of any RGB image at a specific spatial location



Original Color Image



Matlab RGB Matrix

Extracting each matrix of RGB:

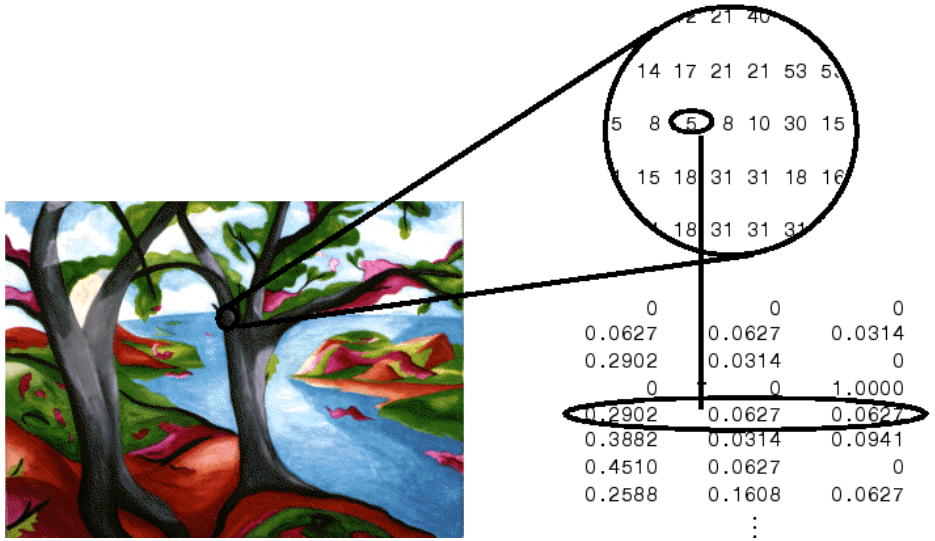
```
>> bluechannel = image[:,:,0]
>> greenchannel = image[:,:,1]
>> redchannel = image[:,:,2]
```

Form an RGB image from three images components RGB:

```
>> A = cv2.merge([bluechannel , greenchannel , redchannel ])
```

Indexed Images (8-Bits):

An indexed image has two components: Data matrix of integers and color map matrix which is an mx3 array of class double [0-1].



Converting Indexed image to gray-scale image?!

Indexed Images (8-Bits):

```
# Read the indexed image
indexed_image = cv2.imread('corn.png', cv2.IMREAD_UNCHANGED)
# Extract the color map
colormap = indexed_image[:, :, -1]
 # Remove the alpha channel
indexed_image = indexed_image[:, :, :3]
 # Apply the color map
color_mapped_image = cv2.applyColorMap(colormap, cv2.COLORMAP_JET)
 # Convert to grayscale
grayscale_image = cv2.cvtColor(color_mapped_image, cv2.COLOR_BGR2GRAY)
# Display the grayscale image
plt.imshow(grayscale_image, cmap='gray')
plt.show()
```

Exercises:

I. Write a PYTHON-code to:

a. Read an image.
b. Show that image.
c. Write the image to disk by a new name.
d. Show additional info about image.
e. Get the size of that image.
f. Show just the first 10 columns with rows from that image.
g. Find the minimum and maximum values of the Image.
h. What is the intensity value of the pixel located at position(4,7)?

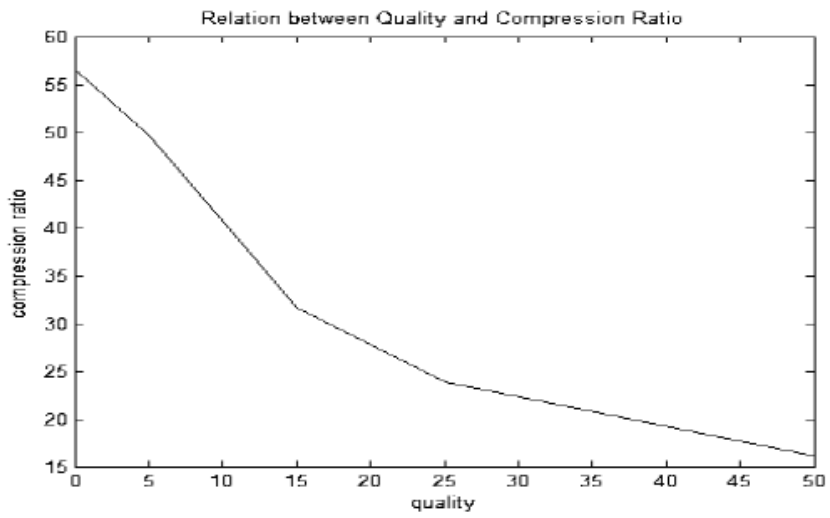
Tip:

Compression Ratio = $\frac{Image\ size\ before\ compression}{Image\ size\ after\ compression}$

II. Plot the relation between Quality and Compression Ratio:

You can follow the following steps:

a. Read the image.
b. Write the image to the disk for q=50, 25, 15, 5, 0.
c. Get the size of the image on hard disk
d. Find the compression ratio for each value of q.
e. Plot the curve as the following one, which shows the relation between quality and compression ratio.



That’s All